Milestone 2 – notatka dla zespołu

Co zrobiliśmy, jak czytać wyniki i o co mogą zapytać

Jerzy Czarnecki, Mateusz Małachowski, Sebastian Rydz, Wojciech Wójcik · 12.11.2026

1. Przypomnienie z Milestone 1 (30 sekund)

- 5574 SMS-y, 13,4% spamu. Metryka główna: **F1 dla klasy spam** (precision = ile z oznaczonych jako spam to naprawdę spam; recall = ile spamu złapaliśmy; F1 łączy oba).
- Zamrożony podział 70/15/15, seed 42. Bliskie duplikaty zawsze w jednej części, więc nie ma przecieku. **train** – nauka, **val** – wybieranie ustawień, **test** – tylko raportowanie.
- Z EDA: spam prawie zawsze ma numer telefonu, kwotę, kod krótki albo link. Stąd hipotezy: **H1/P1** maskowanie numerów i kwot pomoże, **H2/P2** URL: maskowanie bez zmian, usuwanie zaszkodzi, **H3/P3** normalizacja powtórzeń nic nie zmieni na czystym tekście.

2. Co zrobiliśmy w Milestone 2 (w jednym akapicie)

Zbudowaliśmy modele odniesienia (**B0**, reguła **B0r**, **B1**), puściliśmy warianty przetwarzania **P1–P3** na tym samym modelu, połączyliśmy te, które nie szkodziły, i policzyliśmy, czy poprawa jest pewna (**bootstrap**). Dodatkowo sprawdziliśmy wynik na trudniejszych kawałkach testu i po celowym dodaniu literówek. Cały eksperyment jest w `notebooks/02_baselines.ipynb` i trwa kilka sekund. Ustawienia są w `configs/experiments.toml`, a predykcje i tabele w `results/`.

Najważniejsza zasada, którą trzeba umieć powiedzieć: wszystkie decyzje (`C`, próg, który wariant jest „najlepszy") podjęliśmy na **walidacji**. Test policzyliśmy na końcu i niczego na nim nie zmienialiśmy, nawet gdy wyszło, że inny wariant byłby na teście lepszy.

3. Slajd po slajdzie

Slajd 1 – tytułowy

Bez zmian względem M1, tylko „Milestone 2" i data.

Slajd 2 – „B0, reguła B0r i B1: punkty odniesienia"

Punkt odniesienia (*baseline*) to wynik, z którym porównujemy wszystko inne. Mamy trzy:

B0 – klasa większościowa

„Model", który zawsze mówi ham. Ma 86,5% trafień, ale nie łapie ani jednego spamu, więc F1 spamu = 0. Wymaga go regulamin. Pokazuje, że accuracy w tym zadaniu nic nie znaczy.

B0r – reguła ze znaczników (pomysł prowadzącego)

Bez żadnej nauki: SMS jest spamem, jeśli ma **co najmniej jeden** z czterech znaczników z EDA – numer telefonu, kwotę, kod krótki albo URL (`smsspam/rules.py`). „Co najmniej jeden" wybraliśmy na walidacji spośród 1–4. W treningu 87% SMS-ów z jednym znacznikiem i 100% z dwoma lub więcej to spam.

B1 – model bazowy

Surowy tekst → TF-IDF na słowach → regresja logistyczna. Bez czyszczenia; jawne ustawienia: małe litery, domyślny `token_pattern` (token = ≥ 2 litery lub cyfry, interpunkcja wypada), pojedyncze słowa (unigramy). TF-IDF dopasowany tylko do treningu.

- `C` **= 300** – wybrane na walidacji. Duże C = słaba regularyzacja. Kryterium to PR-AUC, a nie F1, bo F1 przy 112 spamach w walidacji mocno skacze. PR-AUC jest prawie takie samo dla każdego C ≥ 10 (0,960–0,962), więc ten wybór niewiele zmienia.
- **Próg decyzji 0,36** – model daje prawdopodobieństwo spamu; powyżej progu mówimy „spam". Próg wybraliśmy na walidacji tak, żeby F1 było najwyższe.

PR-AUC

Pole pod krzywą precision–recall. Mówi, jak dobrze model *szereguje* wiadomości (spam wyżej niż ham), niezależnie od progu. 1 = idealnie; dla losowego szeregowania równa się udziałowi spamu (0,135 – stąd wynik B0). Uzupełnia F1, które zależy od jednego wybranego progu.

Jak czytać tabelę. FP (*false positive*) = ham oznaczony jako spam, czyli fałszywy alarm. FN (*false negative*) = spam, którego nie złapaliśmy.

- B1 na teście: F1 0,940, tylko 1 fałszywy alarm na 724 hamy i 12 przepuszczonych spamów ze 113.
- **Reguła jest prawie tak dobra jak B1**: 0,936 na teście. Różnica +0,3 p.p. z przedziałem [−3,6; +4,2], czyli na teście ich nie odróżnimy. Na walidacji B1 wygrywa wyraźniej (0,922 vs 0,905).
- Gdzie B1 jest lepszy: **mniej fałszywych alarmów** (1 vs 4) i **lepszy ranking** (PR-AUC 0,980 vs 0,915).
- **Wniosek do powiedzenia na głos:** cztery znaczniki z EDA niosą większość sygnału. P1 robi to samo, tylko wewnątrz modelu.

Slajd 3 – „P1–P3: ten sam wektoryzator, to samo C, inny tekst"

Zasada z regulaminu: przy porównaniu przetwarzania **model i ustawienia się nie zmieniają**, zmienia się tylko tekst.

- **P1**: 09061234567 → `_phone_`, £800 → `_money_`, 87121 → `_shortcode_`.
- **P2a**: link → `_url_`. **P2b**: link usunięty.
- **P3**: `sooo` → `soo`, `!!!!` → `!!`.

Dlaczego `_phone_`, a nie `<PHONE>` jak w M1?

Tokenizer B1 wyciąłby z `<PHONE>` zwykłe słowo `phone`, które naprawdę występuje w SMS-ach ("call my phone"). Podkreślnik liczy się jako litera, więc `_phone_` zostaje jednym, osobnym tokenem.

Dlaczego próg jest inny dla każdego wariantu?

W M1 planowaliśmy zamrozić próg z B1. Ale maskowanie zmienia rozkład prawdopodobieństw (np. P1 daje wyższe wyniki spamowi), więc próg B1 byłby dla P1 po prostu źle ustawiony. Próg to reguła decyzji, a nie hiperparametr modelu, i dalej wybieramy go **tylko na walidacji**. Zamrożone są: wektoryzator i `C`. Dla uczciwości w notebooku jest kolumna „F1 @ próg B1" – P1 wygrywa także przy progu B1 (0,954 vs 0,922 na walidacji).

Wyniki pojedynczo (walidacja → test, F1 spamu; B1: 0,922 → 0,940):

- **P1**: 0,959 → 0,959. Najlepszy. Mniej przepuszczonego spamu (FN 12 → 8), bez nowych fałszywych alarmów. Zgodne z H1.

- **P2a**: 0,926 → 0,944. Prawie bez zmian. Zgodne z H2.
- **P2b**: 0,906 → 0,919. Gorzej: model korzysta z kawałków domen (`uk`, `www`). Zgodne z H2.
- **P3**: 0,922 → 0,940 – na teście *dokładnie te same* predykcje co B1. Zgodne z H3.

Połączenie. Łączymy warianty, które na walidacji nie pogorszyły ani F1, ani PR-AUC: P2a, P1, P3 (P2b odpada). Kolejność: najpierw URL, potem numery (żeby cyfry w linku nie zostały zamaskowane jako numer), na końcu powtórzenia. Na walidacji wygrało **P2a+P1+P3**: 0,968, precision 1,000. Na teście: 0,954, 0 fałszywych alarmów, 10 FN.

Pułapka, o którą mogą zapytać: na teście *sam P1* (0,959) wypada lepiej niż wybrane połączenie (0,954). Nie zmieniamy wyboru, bo to byłby dobór na teście – dokładnie to, czego nie wolno. Do tego P2a+P1 i P2a+P1+P3 remisują na walidacji (F1 identyczne, PR-AUC różni się o 0,0001), więc P3 w połączeniu nic nie wnosi.

Slajd 4 – „Czy poprawa jest pewna? Bootstrap sparowany"

Bootstrap sparowany i przedział ufności (CI)

2000 razy losujemy ze zwracaniem nowy „test" z 837 SMS-ów testowych i liczymy F1 obu modeli **na tych samych** wylosowanych SMS-ach. Różnice F1 układają się w rozkład; środkowe 95% to **przedział ufności**. **Jeśli przedział obejmuje 0, nie możemy powiedzieć, że wariant pomaga** – różnica może być przypadkiem.

Jak czytać wykres. Każdy wiersz to jeden wariant. Kropka = różnica F1 względem B1 na całym teście, kreska = 95% przedział. Pionowa linia to zero. Pomarańczowy = wariant wybrany na walidacji. Ciemna kreska: losujemy pojedyncze SMS-y; jasna (zwykle trochę szersza): losujemy całe grupy bliskich duplikatów, bo kopie jednej kampanii nie są niezależne.

- **P1**: +1,9 p.p., CI [−1,0; +5,2] – prawdopodobnie pomaga, ale **nie jest to pewne**. Różnica to 4 złapane spamy więcej, a 1 spam to ok. 0,9 p.p. recall.
- **P2b**: −2,0 p.p., CI [−4,4; −0,5] – jedyny wynik, który jest wyraźny: **usuwanie linków szkodzi**. Przy losowaniu grup przedział dotyka zera, więc i tu ostrożnie.
- **P3**: 0 – kropka bez kreski, bo predykcje są identyczne.
- **Wybrany wariant**: +1,4 p.p., CI [−1,0; +4,1].

To **nie jest porażka**. Regulamin mówi wprost, że brak pewnej poprawy trzeba rzetelnie wykazać, a nie ukrywać. Test ma tylko 113 spamów – za mało, żeby udowodnić różnicę rzędu 2 p.p.

Slajd 5 – „Trudniejsze warunki: podzbiory testu i literówki"

Podzbiory testu (obiecane w M1):

- **bez duplikatów** – z każdego powtórzonego tekstu zostaje jedna kopia (781 SMS-ów). Wyniki prawie te same, więc powtarzane kampanie nie zawyżają wyniku.
- **podobieństwo** $<$ **0,5** – wyrzucamy SMS-y przypominające kampanie z treningu (zostaje 717 SMS-ów, 72 spamy). Oba modele tracą 3–4 p.p. (B1 0,902, wybrany 0,925). Czyli **część wyniku to rozpoznawanie znanych kampanii**. Przewaga wariantu nad B1 zostaje.

Test odporności na literówki

Wymóg tematu. Bierzemy SMS-y z testu i psujemy losowo 10% albo 20% słów (co najmniej 3-literowych) jedną z czterech edycji: usunięcie litery, zamiana dwóch sąsiednich, podmiana na losową, podwojenie (`smsspam/typos.py`). Modele są te same (uczone na czystym tekście), progi też. Losowanie literówek powtarzamy z 3 różnymi ziarnami i podajemy średnią – regulamin wymaga 3 uruchomień dla rzeczy losowych.

- B1: 0,940 → 0,936 (10%) → 0,917 (20%).
- P3: przy 20% 0,922, czyli +0,5 p.p. nad B1. Odchylenie między ziarnami to ok. 0,01, więc **to jest w szumie**.
- Wybrany wariant: 0,954 → 0,949 → 0,947 – **prawie się nie psuje**. Powód: literówki dotyczą liter, a maski numerów i kwot działają na cyfrach, więc ten sygnał przetrwa.

Slajd 6 – „Problemy i plan na Milestone 3"

Problemy:

- Różnice to kilka SMS-ów, stąd szerokie przedziały.
- Reguła jest blisko B1 – nie trzeba tego ukrywać; to argument za P1.
- Na teście sam P1 jest lepszy niż wybrane połączenie – patrz ramka wyżej.
- **Czego model nie łapie** (błędy B1 na walidacji): spam bez numeru i linku (zaczepki randkowe, reklama Burger Kinga) albo z numerem w dziwnej formie (`146tf150p`, `3POUND`). Fałszywe alarmy to hamy ze słowami typu FREE/SMS albo z domeną.

Plan M3:

- **Stabilność P1** – powtórzona walidacja krzyżowa (z grupami) na train+val: wiele podziałów zamiast jednego, żeby zobaczyć, czy P1 pomaga za każdym razem. Bez dotykania testu.
- **P4** – dekodowanie `<#>` (artefakt korpusu NUS z M1) jako analiza wrażliwości.
- **n-gramy słowne vs znakowe** – osobny eksperyment, bo zmienia reprezentację.
- **Model M** – zamrożone embeddingi zdań (gotowa sieć zamienia SMS na wektor znaczenia) + regresja logistyczna. Strojenie tylko na walidacji.

4. Niewygodne pytania, które mogą paść (i co odpowiedzieć)

Protokół

P: Wybraliście P2a+P1+P3, a na teście P1 jest lepszy. Czemu nie bierzecie P1?

O: „Bo wybór musi zapaść na walidacji. Gdybyśmy teraz zmienili zdanie, test przestałby być niezależnym sprawdzianem. Różnica 0,5 p.p. to zresztą mniej niż jeden SMS w F1 i mieści się w szumie."

P: W M1 mówiliście, że próg zamrozicie z B1. Teraz każdy wariant ma swój.

O: „Zmieniliśmy to świadomie: maskowanie przesuwa prawdopodobieństwa, więc wspólny próg krzywdziłby warianty P. Próg dalej wybieramy tylko na walidacji, a `C` i wektoryzator są zamrożone. W notebooku jest też wynik przy progu B1 – wnioski się nie zmieniają."

P: C = 300 to skraj siatki?

O: „Nie, siatka idzie do 3000. Poza tym PR-AUC jest płaskie od C = 10, więc wybór C prawie nic nie zmienia."

P: Czemu C wybieracie po PR-AUC, skoro metryką główną jest F1?

O: „F1 zależy od progu i przy 112 spamach walidacji skacze o kilka punktów od jednego SMS-a. PR-AUC jest stabilniejsze. F1 optymalizujemy przy wyborze progu."

P: Regresja logistyczna jest losowa? Regulamin każe robić 3 uruchomienia.

O: „Solver `liblinear` dla L2 jest deterministyczny: ten sam wynik za każdym razem. Losowe są literówki i bootstrap – literówki mają 3 ziarna, bootstrap ma stałe ziarno 42 i 2000 losowań."

Reguła B0r

P: Reguła jest prawie tak dobra jak model. Po co model?

O: „Na walidacji model jest lepszy, a na teście ma mniej fałszywych alarmów (1 vs 4) i dużo lepszy ranking (PR-AUC 0,98 vs 0,92), więc można przesuwać próg. Reguła tego nie umie. Do tego P1 łączy obie rzeczy i jest lepszy od obu."

P: Czy reguła nie jest dopasowana do danych? Regexy pisaliście, patrząc na dane.

O: „Na treningu, w ramach EDA w M1. Jedyny parametr reguły (ile znaczników) wybraliśmy na walidacji." (To samo pytanie było w notatce z M1 – odpowiedź musi być spójna.)

Koszty błędów

P: Temat każe porównać koszty fałszywych alarmów i niewykrytego spamu.

O: „W filtrze SMS zgubiona wiadomość od człowieka boli bardziej niż przepuszczony spam. Na teście B1 robi 1 fałszywy alarm na 724 hamy, wybrany wariant 0 – i jednocześnie łapie więcej spamu. Reguła robi 4. Pełną analizę z różnymi progami pokażemy w M3/M4."

Liczby na zapas

test (837 SMS-ów, 113 spamów)	F1 spam	precision	recall	PR-AUC	FP / FN
B0	0,000	0,000	0,000	0,135	0 / 113
B0r (reguła)	0,936	0,963	0,912	0,915	4 / 10
B1	0,940	0,990	0,894	0,980	1 / 12
P1	0,959	0,991	0,929	0,982	1 / 8
P2a	0,944	0,990	0,903	0,980	1 / 11
P2b	0,919	0,990	0,858	0,980	1 / 16
P3	0,940	0,990	0,894	0,981	1 / 12
P2a+P1+P3 (wybrany)	0,954	1,000	0,912	0,982	0 / 10

Czas: trening jednego wariantu $<$ 0,05 s, predykcja całego testu ok. 0,01 s, CPU (Apple M5 Pro), bez GPU.

Definicje do przećwiczenia na głos: baseline; FP i FN; próg decyzji; PR-AUC; co robi C; czym jest przedział ufności i co znaczy, że obejmuje zero; dlaczego bootstrap jest sparowany; dlaczego nie wybieramy na teście.

5. Do zrobienia przed prezentacją

1. Każdy przeczytać ramki „Pułapka” i „Dlaczego próg jest inny” – te pytania są najbardziej prawdopodobne.
2. Ustalić, kto mówi który slajd (zalecane 3–5 slajdów; mamy 5 merytorycznych).
3. Otworzyć notebook `02_baselines.ipynb` przed prezentacją, gdyby trzeba było pokazać tabelę lub błędy.

Liczby pochodzą z `notebooks/02_baselines.ipynb` oraz plików w `results/`.